DevOps Interview

200 Real-Time Questions & Answers

Tools · Concepts · Production Troubleshooting

Linux & Shell	Git & GitHub	Docker	Kubernetes	Jenkins / CI-CD
Ansible / Terraform	AWS / Azure / GCP	Prometheus/Grafana	ELK Stack	Production War Stories

Covers:	Senior / Lead DevOps Engineer Level
Format:	Question → Detailed Answer → Pro Tip
Focus:	Real production scenarios & how issues were resolved

■ SECTION 1: Linux & Shell Scripting

Q1. How do you find and kill a process consuming high CPU in production?

A: Use 'top' or 'htop' to identify the PID. Then run 'ps aux | grep ' to confirm. Kill gracefully with 'kill -15 '; if unresponsive use 'kill -9 '. Always check logs before killing to understand root cause.

Pro Tip: *In production, prefer 'kill -15' first; '-9' skips cleanup and can corrupt files or leave locks.*

Q2. A disk is at 100% on a production server — how do you diagnose and fix it?

A: Run 'df -h' to confirm the full filesystem, then 'du -sh /*' and drill down to find large directories. Use 'find / -size +500M' for big files. Common culprits: old Docker images, log files, core dumps. Clear logs carefully, rotate them, and resize volume if needed.

Pro Tip: *Set up logrotate and disk alerting (>80%) proactively to avoid emergency situations.*

Q3. Explain the difference between hard links and soft links.

A: A hard link points directly to the inode — deleting the original file keeps the data accessible via the hard link. A soft (symbolic) link is a pointer to the file path; if the original is deleted the symlink breaks. Hard links cannot span filesystems; symlinks can.

Pro Tip: *Use 'ls -li' to see inode numbers and identify hard links.*

Q4. How do you check which ports are listening on a Linux server?

A: 'ss -tlnp' or 'netstat -tlnp' lists all listening TCP ports with the process name. 'lsof -i :8080' shows what's using a specific port. In containers use 'docker inspect' or 'kubectl exec' to check inside pods.

Pro Tip: *Replace deprecated 'netstat' with 'ss' on modern systems — it's faster and more detailed.*

Q5. What is the difference between /etc/profile, ~/.bashrc, and ~/.bash_profile?

A: /etc/profile is system-wide, executed for login shells. ~/.bash_profile is user-level login shell config. ~/.bashrc is executed for interactive non-login shells (e.g., opening a terminal). Typically ~/.bash_profile sources ~/.bashrc for consistency.

Pro Tip: *Place environment variables in ~/.bash_profile and aliases/functions in ~/.bashrc.*

Q6. How do you schedule a cron job and debug a failing one?

A: Edit with 'crontab -e'. Format: minute hour day month weekday command. Debug by redirecting output: '* * * * * /script.sh >> /tmp/cron.log 2>&1'. Check 'grep CRON /var/log/syslog' for cron daemon logs. Ensure PATH is set inside the script since cron has minimal environment.

Pro Tip: *Always use absolute paths in cron scripts — cron runs with a stripped-down PATH.*

Q7. How do you write a shell script to monitor a service and restart it if down?

A: Use a loop or systemd service with Restart=always. Simple script: check 'systemctl is-active nginx'; if not active, 'systemctl restart nginx' and send an alert via curl to Slack/PagerDuty. Schedule via cron every minute or use systemd watchdog.

Pro Tip: *Prefer systemd's built-in Restart= over custom cron scripts for service reliability.*

Q8. Explain file permissions — how do you set 'rwxr-xr--' in octal?

A: Permissions are owner/group/others. r=4, w=2, x=1. rwx=7, r-x=5, r--=4. So 'rwxr-xr--' = 754. Set with 'chmod 754 file'. Use 'chown user:group file' to change ownership.

Pro Tip: *Use 'chmod -R 755 /var/www' for web server directories and never use 777 in production.*

Q9. What is a zombie process and how do you handle it?

A: A zombie is a process that has completed execution but still has an entry in the process table because the parent hasn't called wait() to read its exit status. You can't kill a zombie (it's already dead). Kill the parent process to clean it up, or wait for the parent to call wait().

Pro Tip: *Persistent zombies indicate a bug in the parent process — escalate to dev team.*

Q10. How do you analyse a high load average that doesn't match high CPU usage?

A: Load average counts runnable + uninterruptible sleeping (I/O waiting) processes. High load with low CPU often means I/O bottleneck. Use 'iostat -x 1' to check disk I/O, 'vmstat 1' for CPU wait (wa column), and 'iotop' to identify the top I/O consuming process.

Pro Tip: *A load average of 1.0 on a 4-core machine is only 25% utilized — always compare with CPU count.*

Q11. How do you extract specific fields from a log file using shell tools?

A: Use 'awk' for field extraction: 'awk '{print $1, $7}' access.log'. Use 'grep' to filter, 'sed' for substitution, 'cut -d: -f1' for delimiter-based columns. For JSON logs, use 'jq'. Combine with 'sort | uniq -c | sort -rn' to find top values.

Pro Tip: *Pipe 'grep ERROR app.log | awk '{print $5}' | sort | uniq -c | sort -rn' to find top error codes quickly.*

Q12. What is the difference between 'nohup', 'screen', and 'tmux'?

A: 'nohup command &' runs a process immune to hangup signals — it keeps running after logout but you can't reattach. 'screen' and 'tmux' are terminal multiplexers that let you create sessions you can detach from and reattach to later. tmux is more feature-rich with splits and scripting.

Pro Tip: *Prefer tmux in production for long-running operations so you can safely detach and reattach.*

Q13. How do you secure SSH on a production Linux server?

A: Disable root login (PermitRootLogin no), disable password auth (PasswordAuthentication no), use key-based auth, change default port (obscurity), allow only specific users (AllowUsers), use Fail2Ban to block brute force, and enable firewall rules to limit SSH source IPs.

Pro Tip: *Use 'ssh-copy-id' to deploy public keys and always test new config with a second session before closing.*

Q14. Explain how 'strace' helps in production debugging.

A: 'strace -p ' traces system calls made by a running process — shows file opens, network calls, memory allocations, and signals. Useful when a process hangs with no logs. 'strace -e trace=network' filters to network calls only. Expect some performance overhead.

Pro Tip: *Use 'strace -c' to get a summary of system call counts and time spent — less invasive.*

Q15. How do you increase open file limits for a process in production?

A: Check current limits with 'ulimit -n'. Set temporarily with 'ulimit -n 65536'. Permanently, edit /etc/security/limits.conf: '* soft nofile 65536' and '* hard nofile 65536'. For systemd services, set 'LimitNOFILE=65536' in the unit file. Verify with 'cat /proc//limits'.

Pro Tip: *Many performance issues (Nginx, Node, Elasticsearch) are caused by hitting the default 1024 file limit.*

■ SECTION 2: Git & Version Control

Q16. What is the difference between git merge and git rebase?

A: Merge creates a new 'merge commit' combining two branches, preserving full history. Rebase moves commits from one branch onto another, creating a linear history but rewriting commit hashes. Use merge for public branches; rebase for cleaning up local feature branches before merging.

Pro Tip: *Never rebase shared/public branches — it rewrites history and causes conflicts for teammates.*

Q17. How do you revert a bad commit that has already been pushed to main?

A: Use 'git revert ' which creates a new commit that undoes the changes — safe for shared branches. Avoid 'git reset --hard' on pushed commits as it rewrites history. After revert, push normally. If rollback needs to be immediate, also redeploy the previous artifact.

Pro Tip: *Always use 'git revert' for production hotfixes — it maintains audit trail.*

Q18. Explain GitFlow vs trunk-based development.

A: GitFlow uses long-lived branches (main, develop, feature/*, release/*, hotfix/*) — good for scheduled releases. Trunk-based development has everyone committing directly to main/trunk with feature flags for incomplete work — enables continuous delivery. Most modern DevOps teams prefer trunk-based.

Pro Tip: *Trunk-based development requires strong CI/CD and feature flags — start with GitFlow if CI/CD isn't mature.*

Q19. How do you handle merge conflicts in a CI/CD pipeline?

A: CI pipelines should fail fast on merge conflicts. Enforce branch protection rules requiring branches to be up-to-date before merging. Developers should frequently pull from main and resolve conflicts locally. In the pipeline, a pre-merge step can detect conflicts and block the PR automatically.

Pro Tip: *Use 'git fetch origin && git merge origin/main --no-commit --no-ff' in CI to detect conflicts early.*

Q20. What is a Git hook and how do you use it in DevOps workflows?

A: Git hooks are scripts triggered at specific points (pre-commit, pre-push, post-merge, etc.). Pre-commit hooks can run linters, formatters, and security scanners before code is committed. Use tools like 'husky' for Node.js or 'pre-commit' framework to manage hooks across teams.

Pro Tip: *Store hooks in the repo and symlink them — raw .git/hooks/ files aren't committed by default.*

Q21. How do you squash commits before merging a feature branch?

A: Interactive rebase: 'git rebase -i HEAD~N' where N is the number of commits. Change 'pick' to 'squash' (or 's') for commits to merge into the previous one. Edit the combined commit message. Then force-push the branch. Alternatively, use 'Squash and merge' in GitHub/GitLab UI.

Pro Tip: *Squashing keeps main branch history clean — one commit per feature makes 'git bisect' effective.*

Q22. How do you recover a deleted branch in Git?

A: Use 'git reflog' to find the last commit hash of the deleted branch, then 'git checkout -b ' to recreate it. Reflog keeps entries for 90 days by default. If the branch was on a remote, check if it still exists: 'git fetch origin '.

Pro Tip: *Always keep reflog retention high (default 90 days) and never run 'git gc' on production repos without a backup.*

Q23. What is a shallow clone and when would you use it in CI/CD?

A: A shallow clone ('git clone --depth 1') fetches only the latest commit without full history, reducing clone time and storage. Ideal for CI/CD pipelines that only need to build the current state. Most CI systems (GitHub Actions, Jenkins) use shallow clones by default for speed.

Pro Tip: *Add '--depth 1 --single-branch' to pipeline clone commands to significantly speed up CI build times.*

Q24. How do you use Git tags in a release pipeline?

A: Tags mark specific commits as releases. Create annotated tags: 'git tag -a v1.2.0 -m "Release v1.2.0"'. Push with 'git push origin v1.2.0'. CI/CD pipelines can trigger deployments on tag events (e.g., GitHub Actions 'on: push: tags: v*'). Semantic versioning (MAJOR.MINOR.PATCH) is the standard.

Pro Tip: *Automate tag creation in CI using tools like 'semantic-release' or 'release-please'.*

Q25. How do you manage secrets in Git repositories?

A: Never commit secrets. Use .gitignore for secret files, pre-commit hooks to scan for secrets (detect-secrets, git-secrets, TruffleHog). If a secret is accidentally committed, rotate it immediately, remove it with 'git filter-branch' or BFG Repo Cleaner, and force-push. Use vault solutions (HashiCorp Vault, AWS Secrets Manager) for secrets management.

Pro Tip: *Run 'git log --all --full-history -- secrets.txt' to audit if a file was ever committed.*

■ SECTION 3: Docker & Containerization

Q26. What is the difference between a Docker image and a container?

A: An image is a read-only template (built from a Dockerfile) containing the application and its dependencies. A container is a running instance of an image — it adds a writable layer on top. Multiple containers can run from the same image simultaneously.

Pro Tip: *Images are immutable; always tag images with version numbers, never rely on 'latest' in production.*

Q27. How do you reduce Docker image size in production?

A: Use multi-stage builds to separate build and runtime environments. Use minimal base images (alpine, distroless). Combine RUN commands to reduce layers. Remove package caches (apt-get clean, rm -rf /var/lib/apt/lists/*). Use .dockerignore to exclude unnecessary files. Scan with 'docker image inspect' and 'dive' tool.

Pro Tip: *A well-optimized production image should be under 100MB — use 'gcr.io/distroless/java' for Java apps.*

Q28. Explain Docker networking modes.

A: Bridge (default): containers on same host communicate via virtual bridge. Host: container shares host's network namespace — no NAT overhead, risky. None: no network. Overlay: multi-host networking for Swarm/Kubernetes. Macvlan: container gets its own MAC address, appears as physical device on network.

Pro Tip: *Use user-defined bridge networks instead of the default bridge — they support DNS resolution by container name.*

Q29. A container keeps restarting in production — how do you debug it?

A: Run 'docker ps -a' to see exit codes. 'docker logs ' for stdout/stderr. 'docker inspect ' for detailed state. Try 'docker run --entrypoint sh ' to override entrypoint and explore the container. Check resource limits with 'docker stats'. Common causes: OOM kill, wrong entrypoint, missing env vars.

Pro Tip: *Exit code 137 = OOM kill (increase memory limit). Exit code 1 = application error (check app logs).*

Q30. What are Docker volumes and when do you use bind mounts vs named volumes?

A: Volumes are for persistent data outside the container lifecycle. Named volumes ('docker volume create') are managed by Docker, portable, and preferred for production databases. Bind mounts map a host path directly into a container — useful for development (hot reload) but less portable and has host filesystem dependency.

Pro Tip: *Never store stateful data inside a container — always use volumes for databases and uploaded files.*

Q31. How does Docker layer caching work and how do you optimize it?

A: Docker caches each RUN/COPY/ADD layer. If a layer's instruction or input files change, all subsequent layers rebuild. Optimization: put rarely-changing instructions first (OS packages), then frequently-changing ones last (application code). Copy package.json before copying source code so npm install is cached.

Pro Tip: *Structure Dockerfile as: base → system deps → app deps (package.json) → app code → CMD.*

Q32. What is a multi-stage Docker build? Give a production example.

A: Multi-stage builds use multiple FROM statements. Earlier stages build artifacts, the final stage copies only what's needed. Example for Go: Stage 1 (golang:alpine) compiles the binary. Stage 2 (scratch or alpine) copies only the binary — final image has no compiler, significantly smaller and more secure.

Pro Tip: *The final stage should have no build tools, package managers, or source code — only runtime artifacts.*

Q33. How do you implement health checks in Docker?

A: Use HEALTHCHECK in Dockerfile: 'HEALTHCHECK --interval=30s --timeout=10s --retries=3 CMD curl -f http://localhost:8080/health || exit 1'. Docker marks container as healthy/unhealthy. Kubernetes uses separate liveness/readiness probes but Docker health checks are still useful for docker-compose and Swarm.

Pro Tip: *Always implement a /health endpoint in your application that checks DB connectivity and critical dependencies.*

Q34. How do you handle secrets in Docker containers?

A: Never use ENV for secrets (visible in 'docker inspect'). Use Docker secrets (Swarm), Kubernetes secrets, or mount secrets as files at runtime. Better: use external secret managers (Vault, AWS SSM) and inject at runtime via entrypoint scripts. Build-time secrets can use BuildKit's --secret mount flag.

Pro Tip: *Run 'docker history ' — if secrets appear in build layers, rebuild with BuildKit secret mounts.*

Q35. What is the difference between ENTRYPOINT and CMD in a Dockerfile?

A: ENTRYPOINT defines the executable that always runs — it's not overridden by 'docker run' arguments. CMD provides default arguments to ENTRYPOINT or the default command if no ENTRYPOINT. 'docker run myimage bash' will override CMD but not ENTRYPOINT. Best practice: use ENTRYPOINT for the application, CMD for default flags.

Pro Tip: *Use exec form (JSON array) for both: ENTRYPOINT ["nginx"] CMD ["-g", "daemon off;"] — avoids shell signal handling issues.*

Q36. Explain Docker Compose and its role in microservices development.

A: Docker Compose defines multi-container applications in a YAML file — services, networks, volumes. 'docker-compose up -d' starts all services with proper dependencies. Used primarily for local development and testing of microservices. Not recommended for production at scale (use Kubernetes instead).

Pro Tip: *Use 'depends_on' with 'condition: service_healthy' to wait for dependencies to be ready before starting a service.*

Q37. How do you set resource limits on Docker containers?

A: Use '--memory' and '--cpus' flags: 'docker run --memory=512m --cpus=1.5 myapp'. In Compose: 'resources: limits: memory: 512M cpus: 1.5'. Without limits, a runaway container can starve the host. In Kubernetes, use 'resources.requests' and 'resources.limits' in pod specs.

Pro Tip: *Always set memory limits in production — an uncapped container causing OOM can crash the entire host.*

Q38. How do you scan Docker images for vulnerabilities?

A: Use Trivy ('trivy image myapp:latest'), Snyk ('snyk container test'), or Docker Scout. Integrate into CI/CD pipeline to fail builds on critical CVEs. Regularly rebuild images to pick up base image security patches. Maintain a minimal attack surface by using distroless or scratch images.

Pro Tip: *Run 'trivy image --severity CRITICAL,HIGH myapp:latest' in CI and block deployments on CRITICAL findings.*

Q39. What is Docker Swarm and how does it compare to Kubernetes?

A: Docker Swarm is Docker's native clustering/orchestration — simpler to set up, built into Docker CLI. Kubernetes is more powerful: better scaling, richer ecosystem, self-healing, rolling updates, service mesh integration. Swarm suits small teams needing simple orchestration; Kubernetes is the industry standard for production at scale.

Pro Tip: *Most organizations migrate from Swarm to Kubernetes as they scale — design your Docker Compose files to be k8s-compatible from the start.*

Q40. A Docker container is consuming too much memory — production scenario.

A: Run 'docker stats' to confirm memory consumption. Check application logs for memory leaks. Use 'docker exec -it top' or 'jmap' for JVM apps. Set memory limits to trigger OOM kill rather than host starvation. Investigate with heap dumps (JVM) or memory profilers. Restart container as temporary fix while fixing the root cause.

Pro Tip: *Set '--oom-kill-disable=false' (default) — let OOM killer restart the container rather than crashing the whole host.*

Q41. How do you do zero-downtime deployments with Docker?

A: Use rolling updates: update containers one-by-one. In Swarm: 'docker service update --update-parallelism 1 --update-delay 30s'. Add health checks so new containers must be healthy before old ones stop. Use blue-green: bring up new version alongside old, switch traffic via load balancer, tear down old.

Pro Tip: *Blue-green deployments require double the resources temporarily but provide the safest cutover.*

Q42. How do you debug a container with no shell (distroless/scratch images)?

A: Use ephemeral debug containers: 'kubectl debug -it --image=busybox --target=' in Kubernetes. For Docker, commit the container state to a new image and add a shell: 'docker commit debug-img && docker run -it --entrypoint sh debug-img'. Or use 'nsenter' on the host to enter the container's namespace.

Pro Tip: *Use 'docker cp :/app/logs /tmp/logs' to extract files from containers without a shell.*

Q43. What is Docker's overlay filesystem and how does it work?

A: Docker uses overlay2 (OverlayFS) by default. It layers the image's read-only layers and a thin read-write layer on top. Reads check upper layer first, then lower. Writes go to the upper layer (copy-on-write). This allows efficient sharing of base image layers across multiple containers.

Pro Tip: *Run 'docker system df' regularly to understand overlay disk usage and reclaim with 'docker system prune'.*

Q44. How do you configure Docker logging for production?

A: Docker supports multiple logging drivers: json-file (default), syslog, journald, fluentd, awslogs. In production, use fluentd or awslogs to ship logs to a central system (ELK, CloudWatch). Configure via 'log-driver' in /etc/docker/daemon.json or per-container with '--log-driver'. Always set max-size and max-file limits.

Pro Tip: *Default json-file driver can fill disk — always set: --log-opt max-size=100m --log-opt max-file=3.*

Q45. How do you implement a private Docker registry?

A: Deploy Harbor (enterprise-grade, with scanning, RBAC) or use Docker Registry (open source). Configure TLS, authentication, and access control. Push images: 'docker tag myapp registry.company.com/myapp:1.0 && docker push'. Pull: 'docker pull registry.company.com/myapp:1.0'. Use imagePullSecrets in Kubernetes for registry authentication.

Pro Tip: *Harbor integrates with Trivy for vulnerability scanning and provides image signing with Cosign.*

■ SECTION 4: Kubernetes (K8s)

Q46. Explain the Kubernetes architecture — control plane vs worker nodes.

A: Control Plane: API Server (single entry point), etcd (distributed key-value store for all cluster state), Scheduler (assigns pods to nodes), Controller Manager (ensures desired state). Worker Nodes: kubelet (communicates with API server, manages pods), kube-proxy (networking/iptables rules), container runtime (containerd/CRI-O).

Pro Tip: *etcd is the most critical component — back it up regularly. Loss of etcd = loss of entire cluster state.*

Q47. What is the difference between a Deployment, StatefulSet, and DaemonSet?

A: Deployment: for stateless apps — manages ReplicaSets, supports rolling updates and rollbacks. StatefulSet: for stateful apps (databases) — provides stable network identities, ordered deployment, and persistent volume per pod. DaemonSet: ensures one pod runs on every (or selected) node — used for monitoring agents, log collectors, network plugins.

Pro Tip: *Never use Deployments for databases — use StatefulSets with persistent volumes.*

Q48. How do liveness, readiness, and startup probes differ?

A: Liveness: if it fails, kubelet restarts the container — detects deadlocks. Readiness: if it fails, pod is removed from Service endpoints — stops traffic to unready pods. Startup: gives slow-starting apps extra time before liveness kicks in — prevents premature restarts during initialization.

Pro Tip: *A misconfigured liveness probe can cause a restart loop — use readiness probes more aggressively than liveness.*

Q49. A pod is stuck in CrashLoopBackOff — how do you debug it?

A: 1) 'kubectl describe pod ' — check Events for error messages. 2) 'kubectl logs --previous' — logs from the crashed container. 3) Check exit code in Events. 4) Try 'kubectl run debug --image= --command -- sleep infinity' to explore the image. Common causes: wrong env vars, missing secrets, OOM, wrong command.

Pro Tip: *Exit codes: 1=app error, 137=OOM/SIGKILL, 139=segfault, 143=SIGTERM. Each tells a different story.*

Q50. What are Kubernetes resource requests and limits? Why do they matter?

A: Requests: minimum resources guaranteed for scheduling (scheduler uses this). Limits: maximum resources the container can use (enforced at runtime). If memory limit exceeded: OOM kill. If CPU limit exceeded: throttled (not killed). Setting only limits without requests = bad scheduling. Always set both.

Pro Tip: *Set requests = 70% of actual average usage. Set limits = 150% of requests for headroom.*

Q51. Explain Kubernetes Services — ClusterIP, NodePort, LoadBalancer, ExternalName.

A: ClusterIP (default): internal cluster access only, stable virtual IP. NodePort: exposes service on each node's IP at a static port (30000-32767) — for development/testing. LoadBalancer: provisions a cloud load balancer — production external access. ExternalName: maps service to a DNS name, no proxying.

Pro Tip: *Use Ingress + ClusterIP services for HTTP traffic instead of LoadBalancer per service — more cost-effective.*

Q52. What is a Kubernetes Ingress and how does it work?

A: Ingress is an API object that manages external HTTP/HTTPS access to services, providing host/path-based routing, TLS termination, and load balancing. Requires an Ingress Controller (Nginx, Traefik, HAProxy) running in the cluster. TLS certs managed via cert-manager with Let's Encrypt.

Pro Tip: *Combine Ingress with cert-manager for automatic TLS certificate provisioning and renewal.*

Q53. How do you scale a deployment in Kubernetes — manual and auto?

A: Manual: 'kubectl scale deployment myapp --replicas=5'. Horizontal Pod Autoscaler (HPA): scales pods based on CPU/memory or custom metrics — 'kubectl autoscale deployment myapp --cpu-percent=70 --min=2 --max=20'. Vertical Pod Autoscaler (VPA): adjusts resource requests. KEDA: event-driven autoscaling (queue depth, etc.).

Pro Tip: *Set HPA min replicas >= 2 for high availability — single replica means any restart causes downtime.*

Q54. What is a Kubernetes ConfigMap and Secret? How do you use them?

A: ConfigMap: non-sensitive configuration data injected as env vars or mounted as files. Secrets: base64-encoded (not encrypted by default!) sensitive data. Mount as env vars (envFrom/env valueFrom) or volume mounts. For real security, use external secret managers (Vault, AWS SSM) with External Secrets Operator.

Pro Tip: *Enable etcd encryption at rest for Secrets — base64 encoding alone is NOT encryption.*

Q55. How does Kubernetes handle rolling updates and rollbacks?

A: Deployments use RollingUpdate strategy by default: gradually replaces old pods with new. Configure 'maxUnavailable' and 'maxSurge'. New pods must pass readiness probes before old ones terminate. Rollback: 'kubectl rollout undo deployment/myapp'. Check history: 'kubectl rollout history deployment/myapp'.

Pro Tip: *Set 'minReadySeconds' to ensure new pods stabilize before the rollout continues — prevents flapping.*

Q56. What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

A: PV: cluster-level storage resource provisioned by admin (or dynamic provisioner). PVC: a request for storage by a pod specifying size and access mode. StorageClass enables dynamic provisioning — when a PVC is created, a PV is automatically provisioned from the cloud provider. Access modes: ReadWriteOnce, ReadOnlyMany, ReadWriteMany.

Pro Tip: *Use ReadWriteMany (NFS/EFS) only when multiple pods need shared storage — RWO is more performant.*

Q57. What are Kubernetes Namespaces and how do you use them for multi-tenancy?

A: Namespaces provide logical isolation within a cluster: separate teams/environments (dev, staging, prod). Apply ResourceQuotas to limit total resources per namespace. Use NetworkPolicies to restrict cross-namespace communication. RBAC roles can be scoped to namespaces for access control.

Pro Tip: *Don't use the default namespace in production — always create dedicated namespaces with resource quotas.*

Q58. Explain Kubernetes RBAC — Role, ClusterRole, RoleBinding, ClusterRoleBinding.

A: Role: defines permissions within a namespace. ClusterRole: cluster-wide permissions. RoleBinding: grants a Role to a user/service account in a namespace. ClusterRoleBinding: grants ClusterRole cluster-wide. Always follow least privilege — give pods ServiceAccounts with minimal permissions, not default ServiceAccount.

Pro Tip: *Audit RBAC permissions with 'kubectl auth can-i --list --as system:serviceaccount:default:myapp'.*

Q59. What is a Kubernetes NetworkPolicy and why is it important?

A: NetworkPolicy controls pod-to-pod and pod-to-external traffic using label selectors. By default all pods can communicate with all pods. NetworkPolicy allows whitelisting: 'only frontend can talk to backend on port 8080'. Requires a CNI plugin that supports NetworkPolicy (Calico, Cilium — Flannel does NOT).

Pro Tip: *Start with a default-deny-all policy per namespace, then add explicit allow rules for required communication paths.*

Q60. How do you handle pod disruption in Kubernetes — PodDisruptionBudget?

A: PodDisruptionBudget (PDB) limits how many pods of a deployment can be unavailable simultaneously during voluntary disruptions (node drain, cluster upgrade). 'minAvailable: 2' ensures at least 2 pods are always running. Critical for maintaining availability during maintenance windows.

Pro Tip: *Always define PDBs for production deployments — without them, a node drain can take down all replicas at once.*

Q61. A node is NotReady in Kubernetes — production scenario walkthrough.

A: 1) 'kubectl describe node ' — check Conditions section. 2) SSH to the node: check kubelet status ('systemctl status kubelet'), disk pressure ('df -h'), memory ('free -m'). 3) Check kubelet logs ('journalctl -u kubelet -n 100'). 4) Common causes: disk full, kubelet crash, network plugin issue, certificate expiry.

Pro Tip: *Set node taints automatically when disk > 85% so pods are evicted before the node becomes unschedulable.*

Q62. How do you do blue-green or canary deployments in Kubernetes?

A: Blue-Green: deploy new version as separate Deployment, switch Service selector to new version label when ready, delete old Deployment. Canary: use two Deployments with weighted traffic split via Ingress annotations (Nginx: 'nginx.ingress.kubernetes.io/canary-weight: 10') or a service mesh (Istio VirtualService). Argo Rollouts automates both strategies.

Pro Tip: *Use Argo Rollouts for production — it automates canary analysis, metrics comparison, and automatic rollback.*

Q63. What is a Kubernetes Operator and when would you build one?

A: An Operator extends Kubernetes with custom resources (CRDs) and controllers that encode operational knowledge (install, upgrade, backup, scale) for complex stateful applications. Use existing operators (Prometheus Operator, PostgreSQL Operator) before building. Build one when you have complex day-2 operations to automate.

Pro Tip: *Use Operator SDK or Kubebuilder to scaffold operators — don't write controller-runtime from scratch.*

Q64. How do you monitor resource usage in a Kubernetes cluster?

A: Metrics Server for basic 'kubectl top nodes/pods'. Prometheus + Grafana for full observability. kube-state-metrics for Kubernetes object metrics. Node Exporter for host metrics. Set up alerts on: pod restarts > N, CPU throttling > 50%, memory usage > 80%, PVC usage > 80%.

Pro Tip: *Use pre-built dashboards: Kubernetes cluster monitoring (ID 3119) and pod monitoring (ID 6417) in Grafana.*

Q65. What are Init Containers and sidecar containers?

A: Init containers run to completion before app containers start — used for setup tasks (DB migration, config download, dependency checks). Sidecars run alongside the main container in the same pod —

used for logging (Fluentd), proxying (Envoy/Istio), secret injection (Vault agent). All containers in a pod share network and volumes.

Pro Tip: *Use init containers to wait for dependencies: 'until nslookup postgres; do sleep 2; done' prevents connection errors on startup.*

Q66. How do you manage Kubernetes manifests at scale — Helm vs Kustomize?

A: Helm: package manager for Kubernetes — templates + values files, versioned charts, lifecycle management (install/upgrade/rollback). Kustomize: overlay-based patching without templates — built into kubectl, better for environment-specific patches. Many teams use both: Helm charts with Kustomize overlays for environment customization.

Pro Tip: *Use Helm for third-party apps (Prometheus, Nginx) and Kustomize for your own application configs.*

Q67. What is Istio/service mesh and when do you need it?

A: Istio injects Envoy sidecar proxies into pods for mutual TLS, observability (traces, metrics), traffic management (weighted routing, retries, circuit breaking), and policy enforcement — all without application code changes. You need it when: you have many microservices, need zero-trust networking, or want advanced traffic control.

Pro Tip: *Istio adds overhead (CPU/memory per sidecar) — evaluate lighter alternatives like Linkerd for simpler use cases.*

Q68. How do you handle pod affinity and anti-affinity in Kubernetes?

A: Pod affinity: schedule pod on nodes where certain pods are running (co-locate frontend with cache). Pod anti-affinity: spread pods across nodes/zones for HA. Use 'requiredDuringScheduling' for hard rules and 'preferredDuringScheduling' for soft rules. Node affinity constrains which nodes pods can run on based on node labels.

Pro Tip: *Always set pod anti-affinity for production deployments to spread replicas across availability zones.*

Q69. What is the Kubernetes eviction mechanism and how do you prevent unwanted evictions?

A: Kubelet evicts pods when node is under resource pressure (memory, disk, PID). Eviction order: BestEffort > Burstable > Guaranteed (based on QoS class). Prevent: set resource requests+limits (Guaranteed QoS), use PriorityClasses for critical pods, set PodDisruptionBudgets, and monitor node pressure proactively.

Pro Tip: *Assign PriorityClass 'system-cluster-critical' to essential infrastructure pods to prevent their eviction.*

Q70. How do you back up and restore an etcd cluster?

A: Backup: 'etcdctl snapshot save /tmp/etcd-backup.db --endpoints=$ENDPOINTS --cacert --cert --key'. Store backup in S3/GCS. Restore: stop API server, restore snapshot: 'etcdctl snapshot restore', restart control plane. Automate daily backups with a CronJob. Velero can also backup entire cluster state.

Pro Tip: *Test your etcd restore procedure in a staging cluster regularly — an untested backup is not a backup.*

Q71. What is Cluster Autoscaler and how does it work?

A: Cluster Autoscaler (CA) automatically adjusts the number of nodes. Scale-up: when pods are unschedulable due to insufficient resources, CA provisions new nodes. Scale-down: when nodes are underutilized and pods can be rescheduled elsewhere, CA terminates nodes after a cool-down period. Integrates with cloud provider ASGs.

Pro Tip: *Set 'cluster-autoscaler.kubernetes.io/safe-to-evict: false' annotation on pods that should not be evicted during scale-down.*

Q72. How do you implement pod security in Kubernetes?

A: Use Pod Security Admission (PSA) — replaced deprecated PodSecurityPolicy. Enforce standards: Privileged, Baseline, or Restricted. Set runAsNonRoot: true, readOnlyRootFilesystem: true, drop ALL capabilities, set securityContext at pod and container level. Use OPA/Gatekeeper for custom policies.

Pro Tip: *Start with Baseline policy and gradually move critical namespaces to Restricted mode.*

Q73. A deployment rollout is stuck — how do you diagnose?

A: 'kubectl rollout status deployment/myapp' shows stuck progress. 'kubectl describe deployment myapp' shows events. 'kubectl get pods' — new pods likely in Pending (insufficient resources) or CrashLoopBackOff. Check: resource quotas exceeded ('kubectl describe resourcequota'), image pull failures, readiness probe failures causing rollout to stall.

Pro Tip: *Set 'progressDeadlineSeconds' on Deployments — default is 600s; after this kubectl marks rollout as failed.*

Q74. How do you manage Kubernetes certificate rotation?

A: kubeadm clusters: certificates expire after 1 year. Check: 'kubeadm certs check-expiration'. Renew: 'kubeadm certs renew all'. Managed Kubernetes (EKS/GKE/AKS) handles control plane certs automatically but check node certs. Set up monitoring to alert 30 days before expiry.

Pro Tip: *Expired certificates are a top cause of cluster outages — add certificate expiry monitoring to your alerting stack.*

Q75. How do you implement GitOps with ArgoCD or Flux?

A: GitOps: Git is the single source of truth for cluster state. ArgoCD watches a Git repo and continuously reconciles cluster state to match. Create Application CR pointing to Git repo + path. Set sync policy to automatic. Any manual change ('kubectl apply') gets reverted. Pair with branch-based environments for dev/staging/prod.

Pro Tip: *Use ArgoCD's ApplicationSet to manage hundreds of microservices from a single config — avoids per-app Application CRs.*

■ SECTION 5: CI/CD, Jenkins & GitHub Actions

Q76. Explain the stages of a robust CI/CD pipeline.

A: 1) Source: code commit triggers pipeline. 2) Build: compile, package (Docker build). 3) Test: unit, integration, security scan. 4) Artifact: push to registry/S3. 5) Deploy to staging: automated. 6) Acceptance tests: smoke/E2E tests. 7) Approval gate (optional). 8) Deploy to production: canary or rolling. 9) Post-deploy: smoke tests, monitoring check.

Pro Tip: *Each stage should fail fast — a fast pipeline (under 10 min) gets more developer adoption.*

Q77. What is the difference between Continuous Integration, Delivery, and Deployment?

A: Continuous Integration: automatically build and test every code push. Continuous Delivery: automatically prepare a deployable artifact, but production deployment requires manual approval. Continuous Deployment: every commit that passes tests is automatically deployed to production — no manual gate.

Pro Tip: *Start with CI, then CD with manual gate, then CD without gate once you have confidence in test coverage.*

Q78. How do you structure a Jenkins pipeline (Jenkinsfile)?

A: Use declarative pipeline: 'pipeline { agent any; stages { stage("Build") { steps { sh "mvn package" } } ... } post { failure { emailext ... } } }'. Use shared libraries for common steps. Store Jenkinsfile in the repo root. Use agent { docker } to run stages in containers for isolation.

Pro Tip: *Never use scripted pipeline for new pipelines — declarative is more readable and has better tooling support.*

Q79. How do you handle Jenkins agent scalability?

A: Use Jenkins Kubernetes plugin to dynamically provision build agents as pods — they spin up on demand and terminate after the build. Define agent templates with resource limits. This eliminates idle agent costs and provides isolation. Jenkins controller (master) just orchestrates; builds run on ephemeral agents.

Pro Tip: *Set 'Pod Template' with specific Docker images per pipeline type (node-agent, maven-agent) for reproducible builds.*

Q80. How do you manage secrets in Jenkins pipelines?

A: Use Jenkins Credentials Store — store secrets as 'Secret text', 'Username/password', or 'Secret file'. Reference in pipeline: 'withCredentials([usernamePassword(credentialsId: 'aws-creds', ...)]) { ... }'. Better: integrate with Vault using Jenkins Vault plugin. Never echo credentials in logs — Jenkins masks them automatically.

Pro Tip: *Rotate Jenkins credentials regularly and audit credential usage with the Credentials Binding plugin logs.*

Q81. A Jenkins build is taking too long — how do you optimize it?

A: Parallelize stages with 'parallel { stage("Unit Tests") {...} stage("Lint") {...} }'. Cache dependencies (Maven local repo, npm cache) using workspace stash or mounted volumes. Use incremental builds. Run only affected tests. Distribute builds across multiple agents. Profile with 'Build Timeline' plugin.

Pro Tip: *Identify the bottleneck first with 'Pipeline Stage View' — don't optimize what isn't slow.*

Q82. How do you implement pipeline-as-code and what are its benefits?

A: Store CI/CD pipeline definition (Jenkinsfile, .github/workflows, .gitlab-ci.yml) in the application repository. Benefits: versioned pipeline changes, code review for pipeline changes, easy onboarding, disaster recovery (rebuild from Git), consistency across branches.

Pro Tip: *Treat your Jenkinsfile like production code — review it, test it, and don't allow direct master edits.*

Q83. Explain GitHub Actions — workflows, jobs, steps, and runners.

A: Workflow: YAML file in .github/workflows/ triggered by events (push, PR, schedule). Jobs: units of work that can run in parallel or sequentially with 'needs'. Steps: individual commands or action calls within a job. Runners: GitHub-hosted (ubuntu-latest) or self-hosted. Actions: reusable step packages from marketplace.

Pro Tip: *Use 'actions/cache' to cache dependencies and 'actions/upload-artifact' to share artifacts between jobs.*

Q84. How do you implement environment-specific deployments in CI/CD?

A: Use separate deployment jobs per environment triggered on different branches or tags: push to 'develop' deploys to dev, merge to 'main' deploys to staging, tag push deploys to prod. Use environment secrets/variables (GitHub Environments, Jenkins multi-branch). Require manual approval for production via environment protection rules.

Pro Tip: *Define GitHub Environments with required reviewers for production — this creates a built-in approval gate.*

Q85. How do you prevent a bad deployment from reaching production?

A: Multi-stage pipeline: unit tests → integration tests → security scan → staging deploy → smoke tests → manual approval → production canary → full rollout. Implement deployment gates: check error rate, p99 latency before proceeding. Automated rollback if post-deploy metrics degrade beyond threshold.

Pro Tip: *The cost of a bad production deployment far exceeds the cost of robust testing — invest in quality gates.*

Q86. What is a release artifact and how do you manage artifact versioning?

A: A release artifact is the deployable package (Docker image, JAR, ZIP) produced by the CI pipeline. Version with semantic versioning or commit SHA. Store in artifact registry (JFrog Artifactory, Nexus, ECR). Tag Docker images with: git commit SHA (immutable reference) + semantic version + 'latest' (mutable). Never overwrite immutable tags.

Pro Tip: *Use commit SHA as the primary artifact identifier in CI/CD — 'latest' is ambiguous and dangerous in production.*

Q87. How do you implement automated testing in CI/CD pipelines?

A: Pyramid: many unit tests (fast), fewer integration tests (medium), few E2E tests (slow). Run unit tests on every commit. Integration tests on PR. E2E on merge to main. Use parallel test execution. Set coverage thresholds (fail if < 80%). Include security tests (SAST with SonarQube, DAST with OWASP ZAP).

Pro Tip: *Test in production-like environments — tests passing on sqlite but failing on PostgreSQL is a common pitfall.*

Q88. How do you handle database migrations in CI/CD?

A: Use migration tools (Flyway, Liquibase, Alembic). Store migrations in version control alongside application code. Run migrations as a pre-deployment step (init container in Kubernetes). Make migrations backward-compatible (additive): never drop columns or rename in the same migration as the

code change. Use feature flags for big schema changes.

Pro Tip: *Apply the expand-contract pattern: add new column → deploy app using both → backfill data → remove old column.*

Q89. What is shift-left security and how do you implement it in pipelines?

A: Shift-left: move security testing earlier in the development cycle. In CI/CD: SAST (static code analysis with SonarQube, Semgrep), dependency scanning (OWASP Dependency-Check, Snyk), secret scanning (TruffleHog, GitLeaks), container scanning (Trivy), IaC scanning (Checkov, tfsec). Fail the build on critical findings.

Pro Tip: *Add a pre-commit hook for secret scanning — catching secrets before they're pushed saves significant remediation effort.*

Q90. How do you implement feature flags in a CD pipeline?

A: Feature flags (LaunchDarkly, Unleash, Flagsmith) decouple deployment from release. Deploy code with the flag disabled → enable for internal users → gradual rollout → full enable → clean up the flag. Enables canary releases without infrastructure changes and instant kill-switches for problematic features.

Pro Tip: *Set a TTL on feature flags — stale flags become tech debt. Clean them up within 2 weeks of full rollout.*

Q91. How do you set up a self-hosted GitHub Actions runner?

A: Go to Repo/Org Settings → Actions → Runners → New self-hosted runner. Download the runner application, configure with token. Register as a service: './svc.sh install && ./svc.sh start'. Use labels to target specific runners. In production, run runners as ephemeral containers (Actions Runner Controller on Kubernetes) for security and scalability.

Pro Tip: *Never share self-hosted runners between public repos and private repos — security isolation risk.*

Q92. How do you cache Docker layers in CI/CD pipelines?

A: GitHub Actions: use 'docker/build-push-action' with 'cache-from: type=gha' and 'cache-to: type=gha,mode=max'. Jenkins: mount Docker daemon socket and use '--cache-from' flag. GitLab CI: use 'DOCKER_BUILDKIT=1' and registry caching. Proper layer ordering in Dockerfile is critical for cache hit rate.

Pro Tip: *Registry-based caching (type=registry) is more effective than GHA cache for large images — persists across runners.*

Q93. Explain the concept of pipeline observability.

A: Monitor your pipelines like production services: track build duration, success/failure rate, queue time, flaky test rate, deployment frequency, lead time for changes (DORA metrics). Use Datadog CI Visibility, Grafana with Jenkins metrics, or GitHub Insights. Alert on pipeline degradation proactively.

Pro Tip: *The 4 DORA metrics (deployment frequency, lead time, MTTR, change failure rate) are the gold standard for CI/CD health.*

Q94. How do you implement multi-region deployments in CI/CD?

A: Deploy sequentially: primary region first, wait for health checks, then deploy to secondary regions. Use circuit breakers — halt multi-region rollout if primary shows errors. Store region-specific config in environment variables. Use global load balancer (Route53, Azure Traffic Manager) to route traffic. Test failover scenarios regularly.

Pro Tip: *Deploy to the region with least traffic first (typically APAC for US companies) to validate before high-traffic regions.*

Q95. How do you manage pipeline permissions and security?

A: Principle of least privilege: pipeline service accounts should only have permissions they need. Separate service accounts per environment (dev SA cannot deploy to prod). Use OIDC/workload identity to authenticate to cloud providers without static credentials. Audit pipeline runs and review permission usage quarterly.

Pro Tip: *Use GitHub's OIDC integration with AWS/GCP to avoid long-lived cloud credentials in your CI/CD system.*

■ SECTION 6: Terraform & Infrastructure as Code

Q96. Explain the Terraform workflow — init, plan, apply, destroy.

A: 'terraform init' downloads providers and modules and sets up backend. 'terraform plan' shows what changes will be made (dry run). 'terraform apply' executes changes. 'terraform destroy' tears down infrastructure. Always review 'plan' output carefully before 'apply'. Use '-auto-approve' only in well-tested automated pipelines.

Pro Tip: *Run 'terraform plan -out=tfplan' and apply the saved plan in CI/CD to ensure plan and apply are identical.*

Q97. What is Terraform state and how do you manage it in teams?

A: State file (terraform.tfstate) maps your config to real infrastructure. In teams: use remote backend (S3 + DynamoDB for locking, Terraform Cloud, GCS). Never commit state to Git — it may contain secrets. Enable state encryption. Use workspaces for environment isolation or separate state files per environment.

Pro Tip: *S3 backend with DynamoDB locking is the most common team setup: 's3://tf-state-bucket/prod/terraform.tfstate'.*

Q98. How do you handle sensitive values in Terraform?

A: Mark variables as 'sensitive = true' — Terraform redacts them from plan output. Never hardcode secrets in .tf files. Use 'vault_generic_secret' data source, AWS SSM/Secrets Manager data sources, or pass via environment variables (TF_VAR_db_password). State still contains sensitive values — encrypt state backend.

Pro Tip: *Use 'sensitive = true' on outputs that expose secrets to prevent leakage in CI/CD logs.*

Q99. How do you structure Terraform code for large projects?

A: Use modules for reusable components (VPC module, EKS module, RDS module). Separate state per environment (dev/staging/prod) and per service. Root modules are environment-specific and call reusable child modules. Use a standard layout: modules/, environments/, variables.tf, outputs.tf. Tools like Terragrunt reduce root module duplication.

Pro Tip: *DRY principle: if you've copy-pasted more than once in Terraform, it's time to create a module.*

Q100. What happens if Terraform state gets out of sync with real infrastructure?

A: State drift: resources were changed outside Terraform. Run 'terraform plan' to see the drift. Fix options: 1) 'terraform import ' to bring manual resources into state. 2) 'terraform state rm ' to remove from state (Terraform won't manage it). 3) Reconcile: let Terraform revert to desired state. Prevent with strict 'no manual changes' policy.

Pro Tip: *Use AWS Config or cloud provider drift detection to alert on manual infrastructure changes.*

Q101. How do you test Terraform code?

A: Static analysis: 'tflint', 'terraform validate', 'checkov'/'tfsec' for security. Unit tests: Terratest (Go) spins up real infrastructure, runs assertions, tears down. Contract tests: validate module outputs. In CI: run validate + lint on PR, run Terratest in dedicated test environment for merges.

Pro Tip: *Use 'terraform plan' with '-detailed-exitcode' in CI: exit code 2 = changes pending, which can gate deployment.*

Q102. How do you manage Terraform provider versions and avoid breaking changes?

A: Pin provider versions in 'required_providers' block with '~>' (pessimistic constraint): '~> 4.0' allows 4.x but not 5.0. Use '.terraform.lock.hcl' (provider lock file) committed to Git for reproducible installs. Run 'terraform init -upgrade' intentionally when updating. Review provider changelogs before major version bumps.

Pro Tip: *Treat provider upgrades like dependency upgrades — test in dev, review changelogs, never auto-upgrade in production.*

Q103. What is 'terraform taint' and 'terraform replace'?

A: 'terraform taint ' (deprecated) marks a resource for recreation on next apply. In modern Terraform use 'terraform apply -replace='. Use case: a resource is broken/corrupted and needs to be destroyed and recreated. Useful for EC2 instances with configuration drift, broken SSL certs, etc.

Pro Tip: *Prefer immutable infrastructure patterns — replace resources rather than patching them in-place.*

Q104. How do you use data sources in Terraform?

A: Data sources fetch information from existing infrastructure (not managed by current state) to use in your config. Examples: 'data.aws_ami.latest' to get the latest AMI ID, 'data.aws_vpc.existing' to reference an existing VPC, 'data.aws_route53_zone.domain' for DNS. They're read-only and refresh on every plan/apply.

Pro Tip: *Use data sources instead of hardcoding IDs — it makes configs portable across accounts and regions.*

Q105. Explain Terraform workspaces — when to use them and their limitations.

A: Workspaces maintain separate state for the same configuration ('terraform workspace new prod'). Good for: testing the same infra config with slight variations. Limitations: same backend bucket (security risk if team access to prod), same code base (harder to have prod-only resources), easy to accidentally apply to wrong workspace.

Pro Tip: *Many teams avoid workspaces for environment separation — use separate directories/repos with different state backends instead.*

Q106. What is Ansible and how does it differ from Terraform?

A: Terraform is declarative infrastructure provisioning (creates/manages cloud resources). Ansible is configuration management and application deployment (configures OS, installs software, deploys apps). They complement each other: Terraform provisions VMs, Ansible configures them. Ansible is agentless — uses SSH. Uses YAML playbooks.

Pro Tip: *Use Terraform for infrastructure, Ansible for configuration management, Docker/K8s for application deployment.*

Q107. How do you handle Terraform plan output in CI/CD and prevent unintended changes?

A: Use 'terraform plan -out=tfplan' and post the plan output to PR comments (Atlantis, Spacelift, or custom GitHub Actions). Require human review for any 'destroy' operations. Set policies in Sentinel (Terraform Cloud) to block non-compliant changes. Use 'OPA' (Open Policy Agent) with custom rules.

Pro Tip: *Atlantis is excellent for team Terraform workflows — plan on PR, apply on merge, with Slack notifications.*

Q108. What is the Terraform resource lifecycle — create_before_destroy, prevent_destroy?

A: 'create_before_destroy = true': creates new resource before destroying old — reduces downtime for resource replacements (useful for certificates, ELB target groups). 'prevent_destroy = true': blocks

'terraform destroy' on critical resources (RDS, S3 buckets). 'ignore_changes': ignores drift for specified attributes (auto-scaling group size).

Pro Tip: *Set 'prevent_destroy = true' on production databases, S3 state buckets, and other critical resources.*

Q109. How do you manage multi-account AWS infrastructure with Terraform?

A: Use AWS provider with 'assume_role' to deploy to different accounts: 'provider "aws" { assume_role { role_arn = var.deploy_role_arn } }'. Separate state backends per account. Use AWS Organizations for account management. Terragrunt simplifies multi-account setups with DRY configurations.

Pro Tip: *Never use root account credentials in Terraform — use cross-account IAM roles with least privilege.*

Q110. How do you perform a Terraform upgrade without downtime?

A: 1) Review upgrade guide for breaking changes. 2) Update required_version and run 'terraform init -upgrade'. 3) Run 'terraform plan' — check if any resources will be recreated due to provider changes. 4) Apply in off-peak hours. 5) Use 'terraform state mv' if resources are renamed in the new provider. Test in dev first.

Pro Tip: *Subscribe to Terraform and provider release notes — major version bumps often have resource schema changes.*

■ SECTION 7: Monitoring, Logging & Observability

Q111. Explain the three pillars of observability.

A: Metrics: numeric measurements over time (CPU%, request rate, error rate) — low cost, great for alerting and dashboards (Prometheus/Grafana). Logs: timestamped records of events — high value for debugging (ELK/Loki). Traces: request journey across services — shows latency hotspots in microservices (Jaeger/Zipkin/OpenTelemetry). Together they provide full system visibility.

Pro Tip: *Start with metrics for alerting, add logs for investigation, add traces when you have multiple services.*

Q112. How does Prometheus collect and store metrics?

A: Prometheus uses pull model — it scrapes HTTP /metrics endpoints at configured intervals. Stores time series in TSDB (time series database) locally. Service discovery (Kubernetes SD, file SD) automatically finds targets. Uses PromQL for querying. Long-term storage via remote write to Thanos, Cortex, or VictoriaMetrics.

Pro Tip: *Push gateway exists for batch jobs that can't be scraped — use it sparingly, it's a monitoring anti-pattern for services.*

Q113. How do you write effective Prometheus alerting rules?

A: PromQL alert example: 'sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) > 0.05' — alert when error rate > 5%. Use 'for: 5m' to avoid flapping. Add labels (severity, team) for routing. Add annotations (summary, description, runbook_url) for context in PagerDuty.

Pro Tip: *Alert on symptoms (high error rate, slow response), not causes (high CPU) — users don't care about CPU.*

Q114. What is Grafana and how do you build effective dashboards?

A: Grafana visualizes metrics, logs, and traces from multiple data sources. Effective dashboards: 1 dashboard per service/team. Top row: key SLIs (error rate, latency, throughput, saturation). Use variables for environment/namespace filtering. Set alert thresholds as visual marks. Version dashboards in Git (Grafonnet or JSON export).

Pro Tip: *Use the RED method for service dashboards: Rate, Errors, Duration. USE method for resources: Utilization, Saturation, Errors.*

Q115. A production service shows high latency — how do you diagnose it?

A: 1) Check dashboards: is it all endpoints or one? 2) Check error rate — latency often accompanies errors. 3) Check downstream dependencies (DB, cache, external APIs) for slowness. 4) Look at traces for the slow request path. 5) Check resource saturation (CPU throttling, connection pool exhaustion). 6) Recent deployments correlation.

Pro Tip: *Start with the outermost layer and work inward — check load balancer, then app, then database.*

Q116. How do you set up the ELK stack (Elasticsearch, Logstash, Kibana)?

A: Logstash/Filebeat (agents on nodes) collect and parse logs → ship to Elasticsearch (indexed storage) → visualize in Kibana. In Kubernetes: use Fluentd or Fluent Bit as DaemonSet to collect pod logs, forward to Elasticsearch. Use index lifecycle management (ILM) to auto-delete old logs and control storage costs.

Pro Tip: *Fluent Bit is much lighter than Fluentd — prefer it for Kubernetes log collection as a DaemonSet.*

Q117. How do you define and measure SLIs, SLOs, and SLAs?

A: SLI (Service Level Indicator): actual measurement (availability = successful requests / total requests). SLO (Service Level Objective): target for the SLI (99.9% availability over 30 days). SLA (Service Level Agreement): contractual commitment with penalty if SLO is missed. Monitor error budget (remaining SLO budget) to balance reliability and new features.

Pro Tip: *If error budget is exhausted, freeze feature deployments until budget recovers — this aligns dev and ops incentives.*

Q118. How do you implement distributed tracing in microservices?

A: Use OpenTelemetry (vendor-neutral SDK) to instrument services. Inject trace context (trace ID, span ID) in HTTP headers (W3C TraceContext standard). Spans are sent to a collector (Jaeger, Zipkin, Tempo). Trace a request from API gateway through all downstream services to find the slow hop.

Pro Tip: *Add baggage propagation for user ID and request context — invaluable for debugging user-specific issues in production.*

Q119. How do you monitor Kubernetes cluster health?

A: kube-state-metrics: K8s object metrics (pod restarts, deployment replicas). Node Exporter: host-level metrics. cAdvisor: container resource metrics. Key alerts: pod restart count > N, deployment not progressing, PVC > 80%, node disk/memory pressure, certificate expiry, API server latency.

Pro Tip: *Deploy Prometheus Operator (kube-prometheus-stack helm chart) for a complete K8s monitoring setup out of the box.*

Q120. Explain on-call practices and incident management.

A: Define escalation paths. Use PagerDuty/OpsGenie for alerting with proper routing rules (severity-based, team-based). Follow incident lifecycle: detect → acknowledge (within SLO) → investigate → mitigate → resolve → post-mortem. Maintain a runbook per alert. Track MTTR (Mean Time to Resolution) as a key metric.

Pro Tip: *Blameless post-mortems improve team culture — focus on systemic fixes, not individual blame.*

Q121. How do you handle alert fatigue in a DevOps team?

A: Audit all alerts — categorize as actionable vs noise. Remove or tune noisy alerts. Set proper thresholds (avoid alerting on spikes < 5 minutes). Use alert grouping and silencing during maintenance. Regularly review on-call tickets — high-frequency alerts need permanent fixes, not repeated acknowledgment.

Pro Tip: *A good on-call rotation averages < 5 pages per night shift — more than that indicates systemic reliability issues.*

Q122. How do you implement log aggregation for a microservices application?

A: Structured logging (JSON format) in all services. Correlation ID injected in all requests and propagated via headers. Collect with Fluent Bit DaemonSet → Elasticsearch or Loki. Include: service name, version, environment, trace ID, user ID, request ID. Never log PII/sensitive data. Set log levels per environment.

Pro Tip: *Use Grafana Loki for cloud-native log aggregation — it's cheaper than Elasticsearch for log storage.*

Q123. What metrics would you monitor for a Node.js application?

A: Application: request rate, error rate, response time (p50/p95/p99), event loop lag, active connections. Process: memory heap used/total, GC duration and frequency, CPU usage. Business: active users, transaction rate, cache hit rate. Use prom-client library to expose custom metrics on /metrics endpoint.

Pro Tip: *Event loop lag > 100ms indicates CPU-bound work blocking the loop — profile with clinic.js or 0x.*

Q124. How do you monitor database performance in production?

A: Key metrics: query latency (p99), queries per second, connections used/max, replication lag, lock wait time, cache hit ratio, slow query log. Use PMM (Percona Monitoring) or CloudWatch RDS Insights. Alert on: replication lag > 30s, connections > 80%, slow queries > threshold.

Pro Tip: *Enable slow query log with threshold 100ms — review weekly to find optimization opportunities before they become incidents.*

Q125. Explain synthetic monitoring and how it complements real user monitoring.

A: Synthetic monitoring: simulated transactions run on schedule from multiple locations (Datadog Synthetics, Pingdom, Checkly) — detects issues before users do. Real User Monitoring (RUM): collects actual user performance data (Core Web Vitals, JS errors) via browser agent. Together: synthetics for proactive detection, RUM for user impact measurement.

Pro Tip: *Run synthetics every minute from multiple global locations — you'll catch regional outages before users tweet.*

Q126. How do you implement infrastructure cost monitoring?

A: Use cloud provider cost tools (AWS Cost Explorer, GCP Billing, Azure Cost Management) with tags on all resources. Tools like Infracost integrate with Terraform to estimate cost of IaC changes in PRs. Set budget alerts. Track cost per service/team/environment. Rightsize underutilized resources automatically with Goldilocks.

Pro Tip: *Tag every resource with: environment, team, service, cost-center — untagged resources should be auto-terminated after 48 hours.*

Q127. What is OpenTelemetry and why is it important?

A: OpenTelemetry (OTel) is a CNCF standard for collecting telemetry data (metrics, logs, traces) with vendor-neutral SDKs and the OTel Collector. Instrument once, send to any backend (Datadog, Jaeger, Prometheus, etc.). Replaces vendor-specific APM agents. Auto-instrumentation for many frameworks requires zero code changes.

Pro Tip: *Adopt OTel now — it's becoming the industry standard and avoids vendor lock-in for observability.*

Q128. How do you handle a cascading failure in a microservices system?

A: Implement circuit breakers (Hystrix, Resilience4j) to stop calling failing services. Bulkhead pattern: isolate thread pools per downstream service. Timeout all external calls. Implement retries with exponential backoff and jitter. Rate limit incoming requests. Use load shedding to prioritize critical traffic. Identify and isolate the root cause.

Pro Tip: *Test resilience with chaos engineering (Chaos Monkey, LitmusChaos) before failures happen in production.*

Q129. How do you implement chaos engineering in production?

A: Start in staging. Define steady state (baseline metrics). Hypothesize: 'if we kill one pod, traffic redistributes in < 30s'. Run experiment (Chaos Mesh, LitmusChaos, Gremlin). Observe. If hypothesis fails, fix the system. Gradually move to production during low-traffic periods with safeguards and automatic rollback.

Pro Tip: *GameDays: schedule regular chaos experiments with the whole team — builds confidence and reveals hidden dependencies.*

Q130. How do you build and maintain runbooks?

A: Runbooks document response procedures for known incidents. Format: title, symptoms, investigation steps (commands to run, dashboards to check), remediation steps, escalation path, post-incident steps. Store in Git (Confluence, PagerDuty runbooks). Link directly from alert annotations. Test runbooks quarterly — stale runbooks are worse than none.

Pro Tip: *The best runbook is one that a sleep-deprived engineer at 3am can follow without making things worse.*

■ SECTION 8: AWS & Cloud Infrastructure

Q131. Explain the AWS Shared Responsibility Model.

A: AWS is responsible for security 'of' the cloud: hardware, networking, hypervisor, managed service infrastructure. Customer is responsible for security 'in' the cloud: OS patching, application security, IAM configuration, data encryption, network firewall rules, compliance. Understanding this boundary is critical for security audits.

Pro Tip: *Many breaches come from misconfigured S3 buckets or IAM — the customer side of the model is the most common failure point.*

Q132. How do you design a highly available AWS architecture?

A: Deploy across multiple Availability Zones (minimum 3). Use Auto Scaling Groups with ALB. Multi-AZ RDS for database. ElastiCache Multi-AZ for caching. S3 for static assets (11 nines durability). Route53 health-based routing. Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO) — design to meet them.

Pro Tip: *Always test your HA architecture — don't discover failure during an actual outage.*

Q133. How does AWS IAM work? Explain roles, policies, and best practices.

A: IAM controls who (identity) can do what (action) on which (resource) under what conditions. Policies are JSON documents attached to users, groups, or roles. Roles are assumed by services/users for temporary credentials. Best practices: least privilege, no root account usage, MFA on all humans, use roles for EC2/Lambda (no access keys), rotate keys regularly, use IAM Access Analyzer.

Pro Tip: *Use 'aws iam simulate-principal-policy' to test IAM permissions before applying to production.*

Q134. What is VPC and how do you design one for production?

A: Virtual Private Cloud: isolated network in AWS. Design: CIDR block large enough for growth (/16). Public subnets (ELB, NAT Gateway, Bastion). Private subnets (EC2, EKS, RDS). Database subnets (RDS with no internet route). Enable VPC Flow Logs. Use NAT Gateway (not instance) for outbound internet from private subnets. Peer VPCs or use Transit Gateway for multi-VPC connectivity.

Pro Tip: *Use /24 subnets per AZ per tier — gives 251 usable IPs, enough for most workloads while keeping routing manageable.*

Q135. Explain S3 storage classes and lifecycle policies.

A: Standard: frequent access, low latency. Standard-IA: infrequent access, lower cost, retrieval fee. One Zone-IA: single AZ. Glacier Instant/Flexible/Deep Archive: archival, increasing retrieval time/decreasing cost. Lifecycle policies automate transitions: Standard → S3-IA after 30 days → Glacier after 90 days → Delete after 7 years. Saves 60-80% on storage costs.

Pro Tip: *Enable Intelligent-Tiering for buckets with unpredictable access patterns — it automates tier transitions.*

Q136. How do you troubleshoot high AWS costs?

A: Use Cost Explorer to identify top services and time of increase. Check for: unused/oversized EC2 instances, unattached EBS volumes, data transfer costs (often overlooked), NAT Gateway costs (optimize traffic routing), old Snapshots, idle load balancers. Use Trusted Advisor and Compute Optimizer for recommendations.

Pro Tip: *Data transfer costs are frequently 20-30% of AWS bills — keep traffic within the same AZ and use VPC endpoints for S3.*

Q137. What is EKS and how does it differ from self-managed Kubernetes?

A: EKS is AWS-managed Kubernetes control plane — AWS handles etcd, API server, upgrades, patches. You manage: worker nodes (or use Fargate), add-ons (CoreDNS, kube-proxy, VPC CNI), RBAC, networking. Integrates natively with AWS IAM (IRSA), ELB, EBS/EFS CSI drivers. More expensive but reduces operational overhead significantly.

Pro Tip: *Use IRSA (IAM Roles for Service Accounts) instead of node instance profiles — gives pod-level AWS permissions.*

Q138. How do you implement auto-scaling on AWS?

A: EC2: Auto Scaling Group with scaling policies (target tracking: keep CPU at 60%, step scaling, scheduled). Application: ALB + ASG for horizontal scaling. EKS: Cluster Autoscaler for nodes, HPA/KEDA for pods. Lambda: scales automatically. RDS: Aurora Serverless v2 scales compute. Use Capacity Providers for ECS.

Pro Tip: *Target tracking scaling policies are simpler and more responsive than step scaling for most use cases.*

Q139. How do you secure an S3 bucket in production?

A: Block all public access (S3 Block Public Access settings). Bucket policy: explicit deny for non-HTTPS. Enable S3 Object Lock for compliance. Enable versioning + MFA Delete. Encrypt at rest (SSE-S3 or SSE-KMS). Access via VPC endpoint. CloudTrail + S3 access logs for audit. Regular access reviews with Access Analyzer.

Pro Tip: *Enable S3 Block Public Access at the account level — prevents any bucket from accidentally being made public.*

Q140. How does AWS CloudFormation compare to Terraform for IaC?

A: CloudFormation: AWS-native, no state file (state in CF service), deep native AWS integration, free, supports drift detection. Terraform: multi-cloud, large provider ecosystem, better modularity, explicit state management, better plan output, wider community. For AWS-only shops: either works; for multi-cloud or greenfield: Terraform preferred.

Pro Tip: *Consider CDK (Cloud Development Kit) as a modern alternative — write infra in Python/TypeScript, compiles to CloudFormation.*

Q141. What is AWS Lambda and when should you use serverless?

A: Lambda runs code without managing servers — scales automatically, pay per invocation. Use for: event-driven processing (S3 events, SQS), scheduled tasks, API backends with variable traffic, microservices. Not ideal for: long-running tasks (15 min max), stateful workloads, high throughput with consistent traffic (costly vs EC2). Cold start latency is a concern for latency-sensitive apps.

Pro Tip: *Use Provisioned Concurrency for Lambda functions in the critical path to eliminate cold starts.*

Q142. How do you implement disaster recovery on AWS?

A: DR strategies (RTO/RPO tradeoff): Backup & Restore (cheapest, slowest). Pilot Light (minimal warm standby, core infra running). Warm Standby (scaled-down full copy). Active-Active Multi-Region (no downtime, most expensive). Use Route53 failover, RDS cross-region replicas, S3 cross-region replication, DynamoDB global tables.

Pro Tip: *Test DR quarterly — many organizations discover their DR plan doesn't work during an actual incident.*

Q143. Explain AWS CloudTrail, Config, and GuardDuty — when do you use each?

A: CloudTrail: API audit logging — who did what, when (security forensics, compliance). Config: resource configuration history + compliance rules — did this resource drift from desired state? GuardDuty: threat detection using ML on CloudTrail/VPC flow logs/DNS logs — identifies suspicious activity (compromised credentials, crypto mining, port scanning).

Pro Tip: *Enable all three in every AWS account from day 1 — retroactive enablement misses pre-existing threats.*

Q144. How do you handle AWS credential rotation and secrets management?

A: Use IAM roles instead of access keys wherever possible (EC2, Lambda, EKS IRSA). For applications: use Secrets Manager with automatic rotation. For CI/CD: use OIDC to generate temporary credentials. If you must use access keys: enable CloudTrail, set 90-day rotation policy, use AWS Credential Report to audit old keys.

Pro Tip: *Run 'aws iam generate-credential-report' weekly to identify stale access keys — rotate or delete immediately.*

Q145. What is AWS Systems Manager (SSM) and how do you use it?

A: SSM is a management service for EC2 and on-prem servers. Key features: Session Manager (SSH without bastion host, all sessions logged), Parameter Store (secrets and config), Run Command (run scripts on many instances), Patch Manager (automated OS patching), Automation (runbook-style automation). Eliminates need for bastion hosts.

Pro Tip: *Use Session Manager instead of SSH — no port 22 exposure, sessions are fully audited in CloudTrail.*

Q146. How do you architect a multi-region active-active application on AWS?

A: Route53 latency/geolocation routing. DynamoDB global tables or Aurora Global Database for data sync. Cognito with cross-region sync for auth. S3 cross-region replication. Regional API Gateways + Lambda or ECS. Use Conflict-free Replicated Data Types (CRDTs) or last-write-wins for conflict resolution. Accept eventual consistency where possible.

Pro Tip: *Start with active-passive (simpler) and move to active-active only when RPO=0 is truly required.*

Q147. How do you optimize AWS Lambda cold starts?

A: Minimize deployment package size (only include required dependencies). Use Provisioned Concurrency for critical functions. Keep functions warm with scheduled pings (last resort). Use Lambda SnapStart for Java (pre-initialized snapshots). Choose runtime: Go/Python have faster cold starts than Java/.NET. Avoid heavy initialization outside handler.

Pro Tip: *Lambda cold starts in Python typically take 200-500ms — usually acceptable unless you're handling real-time APIs.*

Q148. What is AWS Service Control Policy (SCP) and how does it work?

A: SCPs are IAM policies attached to AWS Organizations accounts/OUs that define the maximum permissions boundary. SCPs don't grant permissions — they only restrict what can be delegated. Use to: prevent disabling GuardDuty, enforce region restrictions, require MFA, prevent root account usage, enforce tagging. Applied hierarchically through OU structure.

Pro Tip: *SCPs are your last line of defense — use them to enforce non-negotiable security guardrails across all accounts.*

Q149. How do you implement a Landing Zone for a new AWS environment?

A: Landing Zone: pre-configured, secure multi-account AWS environment. Use AWS Control Tower (managed) or build with Organizations + SSO + GuardDuty + Security Hub + Config + CloudTrail. Standard account structure: Management, Audit, Log Archive, Security, Shared Services, and workload accounts. Apply SCPs for guardrails.

Pro Tip: *AWS Control Tower accelerates Landing Zone setup significantly — use it for new environments unless you have very specific customization needs.*

Q150. A production EC2 instance is unreachable — troubleshooting steps.

A: 1) Check EC2 console: instance state (stopped/terminated/running). 2) Check system status checks (hardware failure) and instance status checks (OS). 3) Check security groups and NACLs — port 22/3389 open from your IP? 4) Check VPC routing. 5) Use Session Manager if agent is running. 6) Get system log via EC2 Actions > Get System Log. 7) Detach volume, attach to another instance, check logs.

Pro Tip: *Enable SSM Agent on all EC2 instances — it's your emergency access when SSH/RDP fails and security groups are locked down.*

■ SECTION 9: Production Incident War Stories & Troubleshooting

Q151. Production database is running slow — complete troubleshooting runbook.

A: 1) Check slow query log for top offenders. 2) Run EXPLAIN on slow queries — look for full table scans. 3) Check active connections and locks ('SHOW PROCESSLIST' in MySQL). 4) Monitor: CPU, IOPS, replication lag. 5) Check for missing indexes. 6) Identify if a new deployment triggered the slowdown (query pattern change). 7) Check autovacuum lag (PostgreSQL). 8) Emergency: kill long-running blocking queries.

Pro Tip: *Keep a query performance baseline — sudden regressions after deployment are almost always index or query plan changes.*

Q152. Production deployment failed midway — half old, half new pods running.

A: 1) Assess: are old pods still serving traffic correctly? 2) Check deployment status: 'kubectl rollout status deployment/myapp'. 3) Decision: rollback ('kubectl rollout undo') or forward (fix the failing new pods). 4) If rolling back, verify old image tag is correct. 5) Investigate new pod failures. 6) Communicate status to stakeholders. 7) Ensure readiness probes caught the issue.

Pro Tip: *Post-incident: ensure deployment validation tests catch the failure before hitting 50% rollout threshold.*

Q153. Memory leak detected in production Java application — response.

A: 1) Confirm with heap metrics trending up without recovery. 2) Take heap dump without restarting: 'jmap -dump:format=b,file=heap.hprof '. 3) Analyze with Eclipse MAT or VisualVM — find the dominators (objects holding most memory). 4) Identify the code path. 5) Short-term: add memory limit + restart policy. 6) Long-term: code fix. 7) Add GC metrics monitoring.

Pro Tip: *Enable JVM flags: -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps — get automatic heap dumps on OOM.*

Q154. DNS resolution failures in production microservices — debugging.

A: 1) 'nslookup ' from inside the pod. 2) Check CoreDNS pods: 'kubectl get pods -n kube-system | grep coredns'. 3) 'kubectl logs -n kube-system '. 4) Check DNS config: 'cat /etc/resolv.conf' inside pod. 5) Test external DNS resolution separately from internal. 6) Check CoreDNS ConfigMap for forwarding rules. 7) Scale up CoreDNS pods if overloaded.

Pro Tip: *CoreDNS is a critical dependency — monitor its request rate, error rate, and latency. Increase replicas proactively.*

Q155. SSL certificate expired in production — emergency response.

A: 1) Verify: 'echo | openssl s_client -connect domain.com:443 2>&1 | grep -A 2 Certificate'. 2) For Let's Encrypt: 'certbot renew --force-renewal'. 3) For ACM (AWS): certificates auto-renew if domain validation is in place — check validation records. 4) For Ingress with cert-manager: 'kubectl describe certificate' and 'kubectl describe challengerequest'. 5) Restart ingress controller after renewal.

Pro Tip: *Monitor certificate expiry 30/14/7 days out — set up automated alerting with Prometheus blackbox exporter.*

Q156. Production Kubernetes cluster is experiencing network partition — response.

A: 1) 'kubectl get nodes' — which nodes are NotReady? 2) Check if control plane is reachable. 3) SSH to affected nodes: check network interfaces, routes, security groups. 4) Check CNI plugin pods: 'kubectl get

pods -n kube-system' (Calico/Flannel/Cilium pods). 5) Check etcd health: 'etcdctl endpoint health'. 6) If split-brain: etcd will refuse writes without quorum (need 3+ members).

Pro Tip: *Run clusters with odd numbers of nodes (3, 5) for etcd quorum — even numbers have no advantage and hurt quorum.*

Q157. Application throwing 'connection refused' errors for downstream service — debug.

A: 1) Is downstream service running? 'kubectl get pods'. 2) Is service endpoint populated? 'kubectl get endpoints '. 3) Is port correct? 'kubectl describe service'. 4) Network policy blocking traffic? 5) Test from inside pod: 'kubectl exec -it -- curl http://downstream:8080/health'. 6) Check downstream service logs for binding errors. 7) Check resource exhaustion causing service restart.

Pro Tip: *Empty endpoints ('kubectl get endpoints') means no pods match the service selector — check label mismatches.*

Q158. Sudden spike in 500 errors after deployment — incident response.

A: 1) Is this a new deployment? 'kubectl rollout history'. 2) What changed? Compare environment variables, image, config. 3) Check application logs for specific error messages. 4) Check error rate by endpoint — all endpoints or specific one? 5) Feature flag kill switch if available. 6) Decision: rollback (fastest) or hotfix. 7) Rollback: 'kubectl rollout undo deployment/myapp'. 8) Post-rollback: verify error rate drops.

Pro Tip: *Decision rule: if error rate > 1% after 5 minutes, rollback immediately — don't wait to investigate while users are impacted.*

Q159. Jenkins pipeline is hung and not completing — troubleshooting.

A: 1) Check if build agent is reachable in Jenkins UI. 2) Look at build logs — where did it stop? Waiting for input? Hung shell command? 3) Check agent resources: disk space, memory. 4) SSH to agent: check running processes. 5) Docker daemon hung? Try 'docker ps'. 6) Increase stage timeouts in Jenkinsfile. 7) Kill the stuck build, restart agent, retry.

Pro Tip: *Add 'timeout(time: 30, unit: 'MINUTES')' wrappers to all pipeline stages — a hung stage should not block the queue forever.*

Q160. Production Redis is at 100% memory — emergency response.

A: 1) Connect: 'redis-cli info memory'. 2) Check top memory consumers: 'redis-cli --bigkeys'. 3) Check TTL usage: 'redis-cli info keyspace' — are keys expiring? 4) Emergency: set maxmemory-policy to allkeys-lru for automatic eviction. 5) Scale Redis instance or add a cluster node. 6) Application-side: audit for keys without TTL. 7) Long-term: set maxmemory with eviction policy from the start.

Pro Tip: *Never run Redis without a maxmemory limit in production — uncontrolled growth leads to OOM and data loss.*

Q161. How do you handle a DDoS attack on your production infrastructure?

A: 1) Enable AWS Shield Advanced or Cloudflare Under Attack mode. 2) WAF: enable rate limiting, geo-blocking if applicable. 3) Increase auto-scaling limits temporarily. 4) Identify attack pattern: IP-based (blacklist at WAF/NLB), application-layer (L7 rules in WAF). 5) Enable CloudFront caching to absorb traffic. 6) Contact ISP/cloud support if volumetric. 7) Document for post-incident.

Pro Tip: *Proactive DDoS prep: implement rate limiting, use CDN, set concurrency limits on Lambda, and have WAF rules ready before an attack.*

Q162. Elasticsearch cluster is red — how do you recover?

A: 1) 'GET /_cluster/health' — number of unassigned shards. 2) 'GET /_cat/shards?v' — which shards are unassigned and why. 3) Common causes: node left cluster (data reallocation), disk full (disk watermark exceeded), too many shards per index. 4) Fix disk: delete old indices or increase watermark temporarily. 5) Reroute shards: 'POST /_cluster/reroute'. 6) Scale cluster if persistent.

Pro Tip: *Set index lifecycle management (ILM) policies from day 1 to auto-delete old indices and prevent disk full scenarios.*

Q163. How do you handle a compromised AWS access key?

A: 1) Deactivate the key immediately in IAM console. 2) Check CloudTrail for all API calls made with the key — last 90 days. 3) Identify any resources created/modified by attacker: EC2 instances, IAM users, S3 data. 4) Terminate attacker resources. 5) Check for backdoor IAM users created. 6) Scan for data exfiltration. 7) Enable GuardDuty if not active. 8) Post-incident: enforce no access keys policy.

Pro Tip: *Time is critical — every minute a compromised key is active, the attacker can create more persistent access.*

Q164. Deployment is stuck waiting for old pods to terminate — what's happening?

A: Likely cause: preStop hooks taking too long, or containers not handling SIGTERM properly. Check 'kubectl describe pod '. Pod goes through Terminating state: preStop hook runs, then SIGTERM sent, then after terminationGracePeriodSeconds (default 30s), SIGKILL. Fix: handle SIGTERM in application, reduce preStop hook time, adjust terminationGracePeriodSeconds.

Pro Tip: *Implement graceful shutdown: catch SIGTERM, stop accepting new requests, drain in-flight requests, then exit.*

Q165. How do you investigate and fix high connection count to PostgreSQL?

A: 1) Check: 'SELECT count(*) FROM pg_stat_activity'. 2) Identify connections by state/query: 'SELECT state, wait_event, query FROM pg_stat_activity'. 3) Kill idle connections: 'SELECT pg_terminate_backend(pid) WHERE state = 'idle' AND ...'. 4) Use PgBouncer connection pooler — reduces connections from hundreds to tens. 5) Application-side: ensure connection pool is properly sized and connections are released.

Pro Tip: *PgBouncer in transaction pooling mode can handle 10,000 app connections with just 100 PostgreSQL connections.*

Q166. What do you do when your CI/CD pipeline starts producing flaky tests?

A: 1) Track flaky test rate over time (test analytics). 2) Quarantine confirmed flaky tests (mark with @Flaky, move to separate suite). 3) Root cause categories: timing issues (add proper waits), shared state (improve test isolation), external dependencies (mock/stub). 4) Run flaky tests in retry mode temporarily. 5) Dedicated sprints to fix flaky tests — they erode CI/CD trust.

Pro Tip: *A test that is flaky is worse than no test — it creates noise and causes developers to ignore CI failures.*

Q167. How do you perform a zero-downtime database migration for a large table?

A: Use pt-online-schema-change (pt-osc) or gh-ost for MySQL, pg_repack for PostgreSQL. These tools: create a shadow table with new schema, copy data in batches, use triggers to sync ongoing changes, then atomically swap the tables. Test with production-sized data first. Monitor replication lag during migration.

Pro Tip: *Pt-osc and gh-ost are production-proven for tables with hundreds of millions of rows — never use ALTER TABLE directly.*

Q168. How do you handle a runaway process consuming all disk I/O in production?

A: 1) 'iotop -a' to identify the top I/O process. 2) 'ionice -c 3 -p ' to reduce its I/O priority. 3) 'kill -STOP ' to temporarily pause it. 4) Investigate why: backup running during peak hours? Unoptimized query dumping large data? Log flooding? 5) Fix root cause. 6) Use cgroups to limit I/O of non-critical processes.

Pro Tip: *Configure I/O scheduling policies in production — database processes should have higher I/O priority than background jobs.*

Q169. How do you debug a service that intermittently returns 503 errors?

A: 1) Check upstream load balancer access logs for frequency and pattern. 2) Check application logs for the timeframe. 3) Correlate with metrics: CPU spikes, GC pauses, database latency spikes? 4) Check connection pool exhaustion. 5) Upstream timeout settings — is 503 from load balancer (backend timeout) or application? 6) Review recent deployments. 7) Add tracing to find the specific code path.

Pro Tip: *Intermittent 503s often indicate resource contention — connection pool exhaustion or thread pool saturation are common causes.*

Q170. Production Nginx returns 502 Bad Gateway — debugging process.

A: 1) Check Nginx error logs: 'tail -f /var/log/nginx/error.log'. 2) Is upstream (application server) running? Check process/pod. 3) Test upstream directly: 'curl http://localhost:8080'. 4) Check upstream timeout settings in nginx.conf (proxy_read_timeout). 5) Connection refused = upstream down. Timeout = upstream too slow. 6) Check upstream application logs for errors.

Pro Tip: *502 = upstream down or refused connection. 504 = upstream timed out. Different causes, different solutions.*

Q171. How do you safely drain a Kubernetes node for maintenance?

A: 1) Cordon the node (mark unschedulable): 'kubectl cordon '. 2) Verify PodDisruptionBudgets won't block: 'kubectl get pdb --all-namespaces'. 3) Drain (evict pods): 'kubectl drain --ignore-daemonsets --delete-emptydir-data'. 4) Perform maintenance. 5) Uncordon: 'kubectl uncordon '. 6) Verify pods reschedule successfully.

Pro Tip: *Always cordon first and verify the cluster can handle reduced capacity before draining — don't drain under peak load.*

Q172. How would you handle a data breach incident involving a production database?

A: 1) Isolate immediately: remove public access, change credentials. 2) Preserve evidence: take RDS snapshot, enable enhanced logging. 3) Identify scope: what data was accessed, by whom, for how long (CloudTrail, DB audit logs). 4) Notify security team and legal (GDPR 72-hour notification requirement). 5) Patch the vulnerability. 6) Notify affected users if PII was exposed. 7) Post-incident: security audit.

Pro Tip: *Have an incident response plan with legal, PR, and security contacts pre-defined — you can't draft it during the breach.*

Q173. How do you troubleshoot a Kubernetes pod that is stuck in Pending state?

A: 'kubectl describe pod ' — check Events section. Common reasons: 1) Insufficient resources (CPU/memory) — 'kubectl describe node' for capacity. 2) No matching nodes (node selector, affinity, taints/tolerations). 3) PVC not bound — check 'kubectl get pvc'. 4) Image pull secret missing. 5) Namespace resource quota exceeded — 'kubectl describe resourcequota'.

Pro Tip: *Pending pods can schedule once cluster autoscaler provisions new nodes — check CA logs if nodes aren't provisioning.*

Q174. How do you handle rolling back a database migration that went wrong?

A: Flyway/Liquibase: use undo migrations or rollback scripts. Best practice: write rollback script before applying migration. If data was modified, restore from pre-migration snapshot (most reliable). For additive migrations (new columns): simply redeploy old application version. For destructive migrations (drops): restore from backup — no other safe option.

Pro Tip: *ALWAYS test rollback procedure in staging before production migration — discovering it's impossible during an incident is catastrophic.*

Q175. How do you implement and test disaster recovery for a Kubernetes cluster?

A: Use Velero for backup: backs up Kubernetes resources and PV data to S3. Backup schedule: 'kubectl apply -f schedule.yaml' — daily backups with retention. Test DR: spin up a new cluster, restore from Velero backup, verify application works. Automate DR testing quarterly. Document RTO (time to restore) — set an SLO for it.

Pro Tip: *An untested DR plan is a false sense of security — schedule quarterly DR fire drills and measure actual RTO.*

■ SECTION 10: Advanced DevOps, Security & Architecture

Q176. What are the DORA metrics and how do you improve them?

A: Deployment Frequency: how often you deploy to production (elite: multiple times/day). Lead Time for Changes: commit to production time (elite: < 1 hour). Change Failure Rate: % deployments causing incidents (elite: < 5%). MTTR (Mean Time to Restore): time to recover from incident (elite: < 1 hour). Improve by: investing in CI/CD automation, testing, monitoring, and reducing batch size.

Pro Tip: *Track DORA metrics quarterly and set improvement targets — they correlate directly with business outcomes.*

Q177. Explain the CAP theorem and its practical implications for distributed systems.

A: CAP theorem: a distributed system can guarantee only 2 of 3: Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (system works despite network partition). Since partitions happen, real choice is CP vs AP. Relational DBs tend to CP; Cassandra, DynamoDB tend to AP. Design your system knowing which you chose.

Pro Tip: *Network partitions always happen eventually — design for partition tolerance and explicitly choose C vs A for each service.*

Q178. What is blue-green deployment and what are its trade-offs?

A: Two identical production environments (Blue=current, Green=new). Deploy to Green, run tests, switch traffic from Blue to Green via load balancer/DNS. Instant rollback = switch back to Blue. Trade-offs: requires 2x infrastructure cost, database schema must be compatible with both versions, session state migration, DNS TTL delays. Best for high-risk deployments.

Pro Tip: *Keep Blue environment for at least 1 hour post-cutover — enough time to detect subtle issues before tearing it down.*

Q179. How do you implement zero-trust security in a DevOps environment?

A: Never trust, always verify. Principles: mutual TLS between all services (Istio/Linkerd), strong identity for workloads (SPIFFE/SPIRE), least privilege access (just-in-time), assume breach mindset. Implementation: service mesh for mTLS, OIDC/IRSA for workload identity, network policies, privileged access management, continuous verification.

Pro Tip: *Start with service mesh for mTLS — it's the highest-impact zero-trust control for microservices environments.*

Q180. What is GitOps and how does it differ from traditional CI/CD?

A: GitOps: Git is the single source of truth for both application code AND infrastructure state. Operators (ArgoCD, Flux) continuously reconcile cluster state to match Git. Traditional CI/CD: pipeline pushes changes to cluster (push model). GitOps: operator pulls changes from Git (pull model). Benefits: audit trail, self-healing, disaster recovery (rebuild from Git), no cluster credentials in CI.

Pro Tip: *GitOps prevents configuration drift — any manual 'kubectl apply' is automatically reverted by the operator.*

Q181. How do you handle stateful applications in Kubernetes?

A: Use StatefulSets (stable network identity, ordered deployment). Persistent volumes with StorageClass for automated provisioning. For databases: use purpose-built operators (PostgreSQL Operator, MySQL Operator). Consider: backup strategy, replication topology, upgrade procedures. For truly stateful apps, evaluate if managed cloud services (RDS, Elasticache) are more appropriate.

Pro Tip: *Managed cloud databases (RDS, CloudSQL) have better HA, backup, and operational tooling than self-managed K8s databases for most teams.*

Q182. Explain service mesh concepts — sidecar proxy, control plane, data plane.

A: Data plane: sidecar proxies (Envoy) injected into each pod — intercept all network traffic, enforce policies, collect telemetry. Control plane (Istiod): configures all proxies via xDS API, issues certificates, implements service discovery. Benefits: mTLS, observability, traffic management — all without application code changes. Cost: added latency (< 1ms) and CPU/memory per sidecar.

Pro Tip: *Start with observability features of Istio before enabling strict mTLS — the transition requires careful certificate management.*

Q183. How do you implement policy as code in DevOps?

A: Open Policy Agent (OPA) with Rego language for general policy decisions. Gatekeeper (OPA + Kubernetes admission webhook) for K8s policies: block privileged containers, enforce label standards, require resource limits. Conftest for testing Terraform/Kubernetes manifests in CI. Sentinel for Terraform Cloud. Shift policy left — validate in CI before applying.

Pro Tip: *Start with 3 Gatekeeper policies: require labels, require resource limits, block privileged containers — high impact, low complexity.*

Q184. How do you implement database sharding and what DevOps considerations are there?

A: Sharding: horizontal partitioning of data across multiple DB instances. DevOps considerations: deployment coordination across shards, schema migrations on all shards (use schema management tools), monitoring per-shard metrics, uneven shard load (hotspot detection), resharding (adding shards without downtime). Use middleware (Vitess for MySQL, Citus for PostgreSQL) to simplify.

Pro Tip: *Avoid manual sharding — use Vitess, Citus, or a managed solution like DynamoDB that handles sharding transparently.*

Q185. What is eBPF and how is it used in DevOps/observability?

A: eBPF (extended Berkeley Packet Filter) allows running sandboxed programs in the Linux kernel without modifying kernel source. Used in DevOps for: Cilium (eBPF-based Kubernetes networking + security), Falco (runtime security monitoring), Pixie (instant Kubernetes observability without instrumentation), Parca (continuous CPU profiling). High performance, low overhead alternative to sidecar-based approaches.

Pro Tip: *Cilium replaces kube-proxy with eBPF — significant performance improvement for high-throughput Kubernetes services.*

Q186. How do you implement progressive delivery?

A: Progressive delivery extends CI/CD with controlled, observable rollouts. Techniques: feature flags for targeted releases, canary deployments with automated analysis (check error rate, latency vs baseline), blue-green, A/B testing. Tools: Argo Rollouts (automated analysis), Flagger (Istio/Linkerd integration), LaunchDarkly (feature flags). Automated rollback if analysis fails.

Pro Tip: *Argo Rollouts with Prometheus analysis provider is the most popular open-source progressive delivery solution for Kubernetes.*

Q187. How do you secure the software supply chain?

A: SLSA (Supply chain Levels for Software Artifacts) framework. Steps: verify source (signed commits, branch protection), build (reproducible builds, SBOM generation), dependency security (lock files, Dependabot, Renovate), artifact signing (Sigstore/Cosign), verify before deploy (verify signature in admission controller). Generate SBOM (Software Bill of Materials) for all releases.

Pro Tip: *Use Cosign to sign Docker images and verify signatures in Kubernetes admission webhooks (Sigstore policy controller).*

Q188. What is a Service Level Agreement and how do you design for it?

A: SLA: contractual commitment to a customer (99.9% uptime = 8.7 hours downtime/year). Design backwards from SLA: what SLO do you need? (SLO should be tighter than SLA — 99.95% internal SLO for 99.9% SLA). What SLIs do you measure? What infrastructure redundancy achieves this availability? Calculate error budget. Set up automated SLA reporting.

Pro Tip: *99.9% SLA = 8.7 hours/year downtime. 99.99% = 52 minutes/year. Each nine costs significantly more to achieve.*

Q189. How do you implement cost optimization as code?

A: Infracost in CI: comment estimated cost on every Terraform PR. Cloud Custodian: policy-as-code for cloud resources (terminate idle EC2, delete untagged resources). Goldilocks: VPA-based rightsize recommendations. Spot instances for non-critical workloads (60-80% savings). Reserved Instances/Savings Plans for stable workloads. FinOps culture: engineers see their costs.

Pro Tip: *Show engineers the cost of their infrastructure in dashboards — cost visibility drives cost-conscious architecture decisions.*

Q190. Explain the concept of immutable infrastructure.

A: Immutable infrastructure: never modify servers after deployment — always replace. Old approach: SSH in and patch. Immutable: build new AMI/image with changes, deploy new servers, terminate old. Benefits: consistent environments (no configuration drift), easier rollback (redeploy old image), security (no SSH access needed). Enabled by: Docker, Packer, cloud ASGs.

Pro Tip: *Immutable infrastructure + GitOps = full auditability. Every change is a code change with a PR, review, and Git history.*

Q191. How do you manage technical debt in a DevOps context?

A: Track tech debt in the backlog with clear business impact. Set aside 20% of sprint capacity for debt reduction. Metrics: deployment frequency (low = high debt), build time (long = poor modularity), MTTR (high = poor observability). Prioritize: security debt (urgent), performance debt (when hitting limits), reliability debt (when causing incidents).

Pro Tip: *Refactor continuously in small increments — waiting for a 'big refactor sprint' means it never happens.*

Q192. How do you implement multi-tenancy in a shared Kubernetes cluster?

A: Namespace-per-tenant with ResourceQuotas and LimitRanges. RBAC: tenant-specific service accounts with namespace-scoped roles. NetworkPolicies: default-deny-all + explicit inter-namespace rules. Node isolation for sensitive tenants: dedicated node pools with taints/tolerations. Hierarchical Namespace Controller (HNC) for namespace trees. Soft multi-tenancy (trust boundary exists, defense in depth).

Pro Tip: *Hard multi-tenancy (truly untrusted tenants) in Kubernetes is very difficult — use separate clusters for high-security isolation requirements.*

Q193. How do you implement API versioning in a microservices deployment pipeline?

A: URL versioning (/v1, /v2) or header versioning. Run multiple versions simultaneously during transition. CI/CD: build and deploy all supported API versions. API gateway routes requests to correct version. Deprecation strategy: announce N versions in advance, add deprecation headers, monitor usage, sunset after adoption drops to zero. Document breaking vs non-breaking changes.

Pro Tip: *Semantic versioning for APIs: major bump for breaking changes, minor for new features, patch for bug fixes — communicate this to consumers.*

Q194. What is the twelve-factor app methodology and how does it apply to DevOps?

A: 12-factor principles for cloud-native apps: 1) Codebase in version control. 2) Explicit dependency declaration. 3) Config in environment variables (not code). 4) Backing services as attached resources. 5) Build/release/run separation. 6) Stateless processes. 7) Port binding. 8) Concurrency via processes. 9) Disposability (fast startup, graceful shutdown). 10) Dev/prod parity. 11) Logs as event streams. 12) Admin processes.

Pro Tip: *12-factor apps are naturally containerizable and CD-friendly — use it as the design checklist for new services.*

Q195. How do you implement security scanning in a DevOps pipeline?

A: SAST (Static Application Security Testing): SonarQube, Semgrep, Checkmarx — scans code. DAST (Dynamic): OWASP ZAP against running app. SCA (Software Composition Analysis): Snyk, OWASP Dependency-Check — scan dependencies. Container: Trivy, Grype. IaC: Checkov, tfsec. Secret scanning: TruffleHog, GitLeaks. Integrate all in CI — fail on critical, alert on high.

Pro Tip: *Prioritize: fix Critical first (CVSS > 9), then High (CVSS > 7) — medium/low can be tracked in backlog.*

Q196. How do you design for observability from the start in a new service?

A: Design decisions: structured logging (JSON), meaningful log levels, correlation IDs in all requests, expose /metrics in Prometheus format, expose /health and /ready endpoints, implement trace context propagation, emit business metrics (not just technical), document dashboards and runbooks. Treat observability as a feature requirement, not an afterthought.

Pro Tip: *The cost of adding observability after the fact is 10x the cost of building it in from the start.*

Q197. What is site reliability engineering (SRE) and how does it relate to DevOps?

A: SRE (Google origin) applies software engineering principles to operations: SLOs, error budgets, toil reduction, automation. DevOps is a cultural philosophy; SRE is an implementation of DevOps principles with specific practices. Key SRE concepts: error budget gates deployments, toil tracking (manual repetitive work < 50% of time), blameless postmortems, capacity planning.

Pro Tip: *Adopt SRE practices incrementally — start with SLIs/SLOs and error budgets; they transform the relationship between dev and ops teams.*

Q198. How do you evaluate whether to build or buy a DevOps tool?

A: Build if: highly specific requirements, competitive advantage, existing tools don't fit. Buy/use open source if: commodity function, good existing options, significant build cost. Framework: 1) Define requirements clearly. 2) Evaluate top 3 tools against requirements. 3) Prototype with real workloads. 4) Consider: total cost of ownership, vendor lock-in, community/support, integration complexity.

Pro Tip: *'Build' underestimates maintenance cost — account for ongoing engineering time, not just initial build time.*

Q199. How do you handle configuration management for hundreds of microservices?

A: Centralized config: AWS AppConfig, HashiCorp Consul, Kubernetes ConfigMaps with External Secrets Operator. Hierarchical override: defaults → environment-specific → service-specific. Hot reload: services watch for config changes without restart. Config validation in CI: prevent invalid configs from reaching production. GitOps for config: all configs in Git with PR-based changes.

Pro Tip: *External Secrets Operator + AWS SSM/Secrets Manager is a production-proven pattern for Kubernetes secret management at scale.*

Q200. If you were starting a new startup's DevOps setup from scratch, what would you build?

A: Phase 1 (MVP): GitHub + GitHub Actions (CI/CD), Docker + ECS Fargate or single EKS cluster, Terraform for infra, RDS PostgreSQL, basic Datadog/Grafana Cloud, Route53, ACM certs, basic WAF. Phase 2 (Growth): Kubernetes + Helm/ArgoCD, HashiCorp Vault, PagerDuty, full ELK/observability, multi-AZ, multi-region DR. Phase 3 (Scale): service mesh, chaos engineering, FinOps, SRE practices, multi-cluster.

Pro Tip: *Start simple, evolve with your scale. Over-engineering before product-market fit kills more startups than under-engineering.*

QUICK REFERENCE CHEAT SHEET

Kubernetes Debug	kubectl describe pod/node | kubectl logs --previous | kubectl exec -it | kubectl get events
Docker Debug	docker stats | docker logs -f | docker exec -it | docker inspect | docker system df
Linux Perf	top/htop | iostat -x 1 | vmstat 1 | ss -tlnp | iotop | strace -p
Git Emergency	git reflog | git revert | git stash | git log --oneline | git bisect
Terraform	init → validate → plan -out=tf.plan → apply tf.plan | state list | import | taint
Jenkins	Jenkinsfile in repo | declarative pipeline | parallel stages | withCredentials | timeout()
AWS Quick	aws sts get-caller-identity | aws ec2 describe-instances | aws logs tail | aws ssm start-session
Prometheus	rate()[5m] | increase() | histogram_quantile(0.99,...) | absent() | predict_linear()
Incident Cmd	WHO is affected | WHAT is broken | WHEN did it start | WHY (recent change?) | HOW to fix
DORA Targets	Freq: daily+ | Lead Time: <1hr | CFR: <5% | MTTR: <1hr = Elite performance